



ÉCOLE CENTRALE CASABLANCA

FROM PYTHON ETL TO DBT & SNOWFLAKE SCHEMA
RAPPORT

Modern Data Pipeline Engineering

Élèves :

Falleiz Rosteim BELEMCOABGA
Vincent MENDY

Encadrants :

Lamia BEN HIBA

22 février 2026

Table des matières

1	Introduction	2
1.1	Context	2
1.2	Approach Evolution	2
2	Phase 1 : The Python ETL Pipeline	3
2.1	Overview	3
2.2	Project Architecture	3
2.3	Data Extraction	3
2.3.1	Application Metadata (<code>extract_metadata.py</code>)	3
2.3.2	User Reviews (<code>extract_reviews.py</code>)	3
2.4	Data Transformation	4
2.4.1	Cleaning (<code>transform_data.py</code>)	5
2.4.2	Serving Layer (<code>create_serving_layer.py</code>)	5
2.5	Interactive Dashboard (Streamlit)	6
2.5.1	Overview and Filters	6
2.5.2	Best and Worst Applications (Q1)	7
2.5.3	Rating Trend Over Time (Q2)	7
2.5.4	Review Volume Distribution (Q3)	8
2.6	Limitations of this Approach	9
3	Phase 2 : Modern ELT with dbt & DuckDB	10
3.1	From ETL to ELT : A Paradigm Shift	10
3.2	Data Architecture : From Star to Snowflake Schema	10
3.3	Staging Layer	11
3.4	Mart Layer : Dimensional Models	11
3.4.1	Normalized Dimension Tables	11
3.4.2	Fact Table : <code>fact_reviews</code>	12
3.5	Features Added Beyond Base Requirements	12
3.6	Advanced Features	12
3.6.1	Slowly Changing Dimensions Type 2 — SCD2	12
3.6.2	Incremental Loading — <code>fact_reviews</code>	13
3.7	Data Quality Enforcement	14
4	Comparative Analysis & Critical Reflections	15
4.1	Python-Only vs dbt-Based Pipeline	15
4.2	Critical Reflections	16
4.2.1	Most Fragile Part of the Pipeline	16
4.2.2	Biggest Architectural Insight	16
4.2.3	One Design Decision We Would Change	16
5	Conclusion	17
A	Project Evidence	18
A.1	SCD2 Verification	18
A.2	Incremental Loading Log	18

1 Introduction

1.1 Context

The Google Play Store operates as a colossal marketplace hosting millions of applications, generating massive amounts of data daily. For data analysts and business stakeholders, extracting meaningful insights from this ecosystem, such as trending categories, user sentiment, and application performance, is crucial for strategic decision-making. However, the raw data available often comes in the form of massive, unstructured JSON files, which are inherently difficult to query and analyze directly.

Project Objectives The primary objective of this project, spanning two distinct laboratory sessions, was to architect and implement a robust [Data Pipeline](#) capable of **ingesting** raw metadata and review files, **transforming** this data into a structured analytical model, and finally **serving** actionable insights through interactive dashboards.

1.2 Approach Evolution

Our approach evolved significantly throughout the project. We initially developed a traditional ETL pipeline using Python scripts during the first phase, which allowed us to understand the fundamental logic of data extraction and cleaning. However, recognizing the limitations of this manual approach in terms of scalability and maintainability, we transitioned in the second phase to a modern ELT architecture. This new architecture leverages the power of DuckDB for high-performance compute and dbt (data build tool) for modular, version-controlled transformations, introducing advanced engineering concepts such as Slowly Changing Dimensions (SCD2) and Incremental Loading. This report details the implementation of both phases, highlighting the architectural shift and the tangible benefits of adopting a modern data stack.

2 Phase 1 : The Python ETL Pipeline

2.1 Overview

In the first phase of this project, we developed a complete **ETL (Extract, Transform, Load)** pipeline in Python to analyze the market of AI-powered note-taking applications on the Google Play Store. The objective was to automate data collection, clean and aggregate it, and deliver an interactive dashboard answering specific business questions.

Business Questions Addressed The dashboard answers three core analytical questions :
(1) Which applications perform best or worst based on user ratings? **(2)** Are user ratings improving or declining over time? **(3)** Are there noticeable differences in review volume between competing applications?

2.2 Project Architecture

The project adopts a clear separation of concerns between data processing scripts (`src/`) and the visualization layer (`app/`).

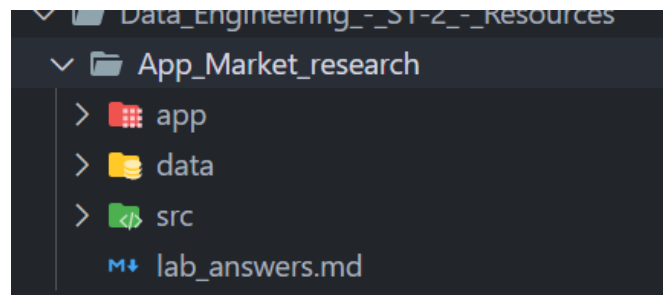


FIGURE 1 – Project directory structure (App_Market_research)

The execution flow is sequential and must be orchestrated manually in the following order : `extract_metadata.py` → `extract_reviews.py` → `transform_data.py` → `create_serving_layer.py` → `app/main.py`

2.3 Data Extraction

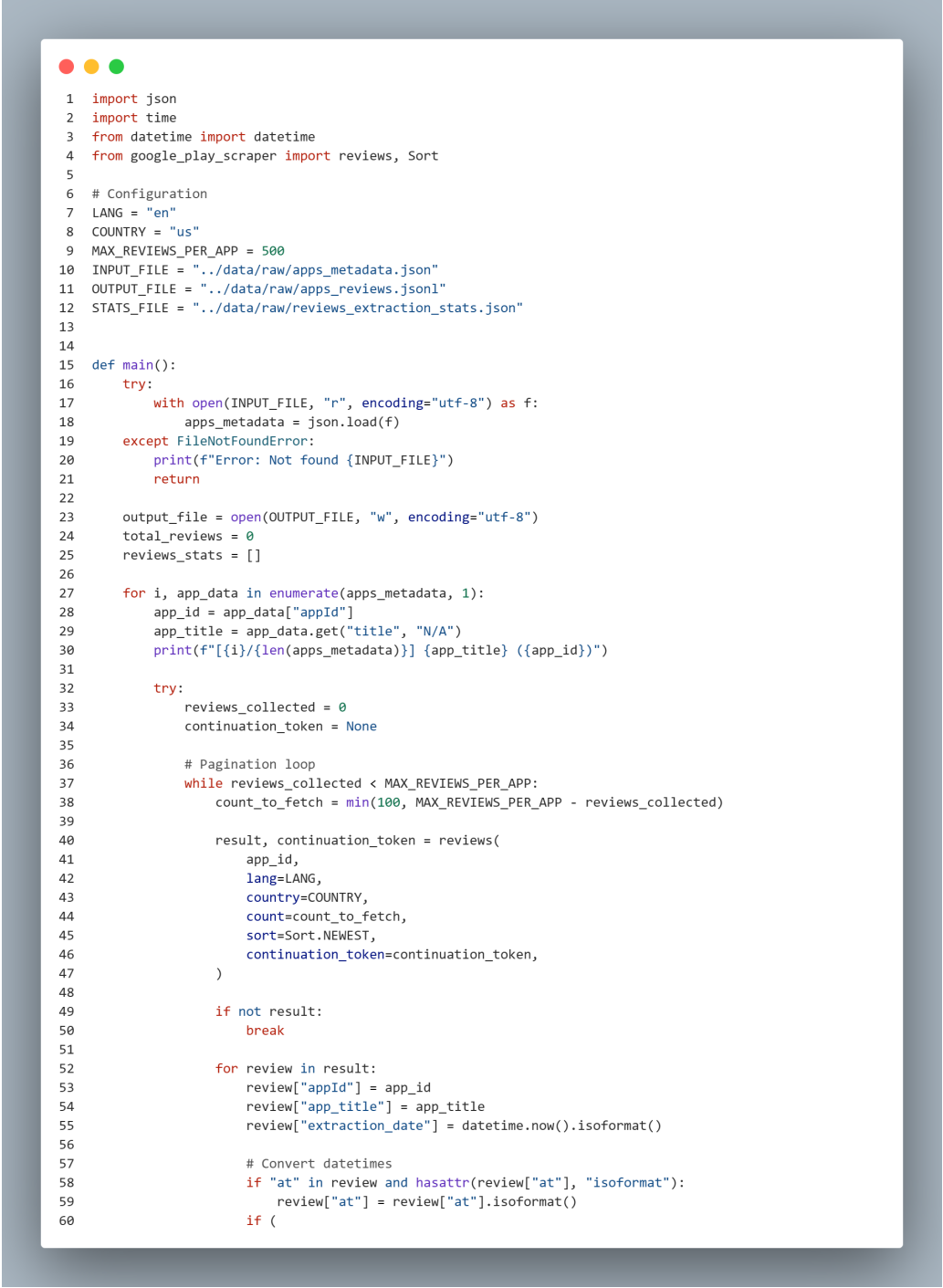
2.3.1 Application Metadata (`extract_metadata.py`)

This script queries the Play Store API via the `google_play_scraper` library to retrieve application details such as score, price, genre, and install count. A `time.sleep(1)` delay is applied between each request to respect API rate limits and avoid being blocked. Applications that fail during extraction are logged to `apps_metadata_errors.json`, enabling partial reruns without reprocessing successful records.

2.3.2 User Reviews (`extract_reviews.py`)

The script collects up to **500 reviews per application**. To overcome the API's per-page limit, it implements a pagination loop using a `continuation_token` returned by

each API call. Reviews are written line by line in **JSONL format** (one JSON object per line), enabling streaming processing without loading all data into memory simultaneously.



```

1  import json
2  import time
3  from datetime import datetime
4  from google_play_scraper import reviews, Sort
5
6  # Configuration
7  LANG = "en"
8  COUNTRY = "us"
9  MAX_REVIEWS_PER_APP = 500
10 INPUT_FILE = "../data/raw/apps_metadata.json"
11 OUTPUT_FILE = "../data/raw/apps_reviews.jsonl"
12 STATS_FILE = "../data/raw/reviews_extraction_stats.json"
13
14
15 def main():
16     try:
17         with open(INPUT_FILE, "r", encoding="utf-8") as f:
18             apps_metadata = json.load(f)
19     except FileNotFoundError:
20         print(f"Error: Not found {INPUT_FILE}")
21         return
22
23     output_file = open(OUTPUT_FILE, "w", encoding="utf-8")
24     total_reviews = 0
25     reviews_stats = []
26
27     for i, app_data in enumerate(apps_metadata, 1):
28         app_id = app_data["appId"]
29         app_title = app_data.get("title", "N/A")
30         print(f"[{i}/{len(apps_metadata)}] {app_title} ({app_id})")
31
32         try:
33             reviews_collected = 0
34             continuation_token = None
35
36             # Pagination loop
37             while reviews_collected < MAX_REVIEWS_PER_APP:
38                 count_to_fetch = min(100, MAX_REVIEWS_PER_APP - reviews_collected)
39
40                 result, continuation_token = reviews(
41                     app_id,
42                     lang=LANG,
43                     country=COUNTRY,
44                     count=count_to_fetch,
45                     sort=Sort.NEWEST,
46                     continuation_token=continuation_token,
47                 )
48
49                 if not result:
50                     break
51
52                 for review in result:
53                     review["appId"] = app_id
54                     review["app_title"] = app_title
55                     review["extraction_date"] = datetime.now().isoformat()
56
57                     # Convert datetimes
58                     if "at" in review and hasattr(review["at"], "isoformat"):
59                         review["at"] = review["at"].isoformat()
60                     if (

```

FIGURE 2 – Pagination loop in `extract_reviews.py` using `continuation_token`

2.4 Data Transformation

2.4.1 Cleaning (transform_data.py)

The `clean_installs` function illustrates the complexity of cleaning raw Play Store data : values such as "1,000,000+" must be converted to integers by stripping commas and the "+" character before type casting.



```
1 import json
2 import pandas as pd
3
4
5 def clean_installs(val):
6     if pd.isna(val):
7         return None
8     return (
9         int(str(val).replace(",", "").replace("+", ""))
10        if str(val).replace(",", "").replace("+", "").isdigit()
11        else None
12    )
13
14
15 def main():
16     print("Transforming data...")
17
```

FIGURE 3 – `clean_installs` function — handling heterogeneous installs formats

2.4.2 Serving Layer (create_serving_layer.py)

Rather than querying raw data directly from the dashboard, we introduced a pre-aggregation step that generates two optimized CSV files. The `pct_low_rating` metric (percentage of reviews with score ≤ 2) is a custom business KPI designed to identify applications with chronic quality issues.



```
1 import pandas as pd
2
3
4 def main():
5     print("Creating Serving Layer...")
6
7     # Load data
8     try:
9         reviews_df = pd.read_csv("../data/processed/apps_reviews.csv")
10        reviews_df["at"] = pd.to_datetime(reviews_df["at"])
11    except FileNotFoundError:
12        print("Error: apps_reviews.csv not found.")
13        return
14
15    # 1. App-Level KPIs
16    # Metrics: count, avg rating
17
```

FIGURE 4 – pct_low_rating KPI calculation in create_serving_layer.py

2.5 Interactive Dashboard (Streamlit)

The final deliverable of the pipeline is an interactive dashboard built with [Streamlit](#) and [Plotly](#), consuming the pre-aggregated CSV files from the serving layer.

2.5.1 Overview and Filters

The main page displays a global metrics panel showing the total number of tracked applications, the total review count, and the global average rating. A sidebar provides a multi-select filter that allows comparing a custom subset of competing applications.

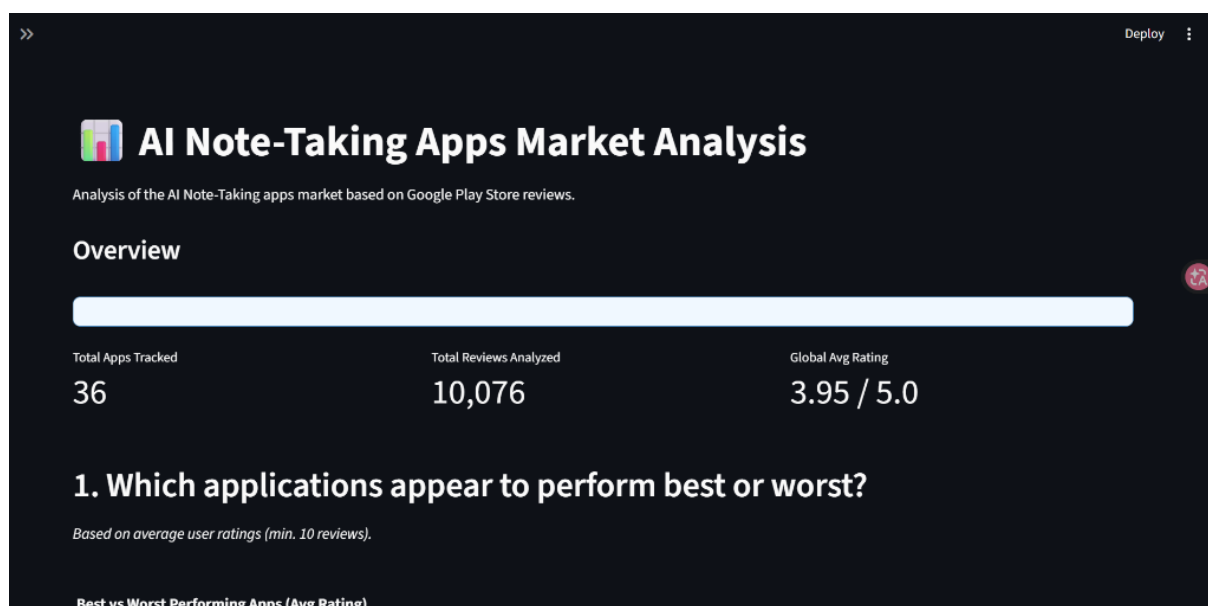


FIGURE 5 – Dashboard — Overview panel with global metrics and sidebar filters

2.5.2 Best and Worst Applications (Q1)

The first chart answers Question 1 by displaying a color-coded horizontal bar chart showing the Top 5 and Bottom 5 applications by average rating. Only applications with at least 10 reviews are included to avoid statistical bias from low-volume entries.

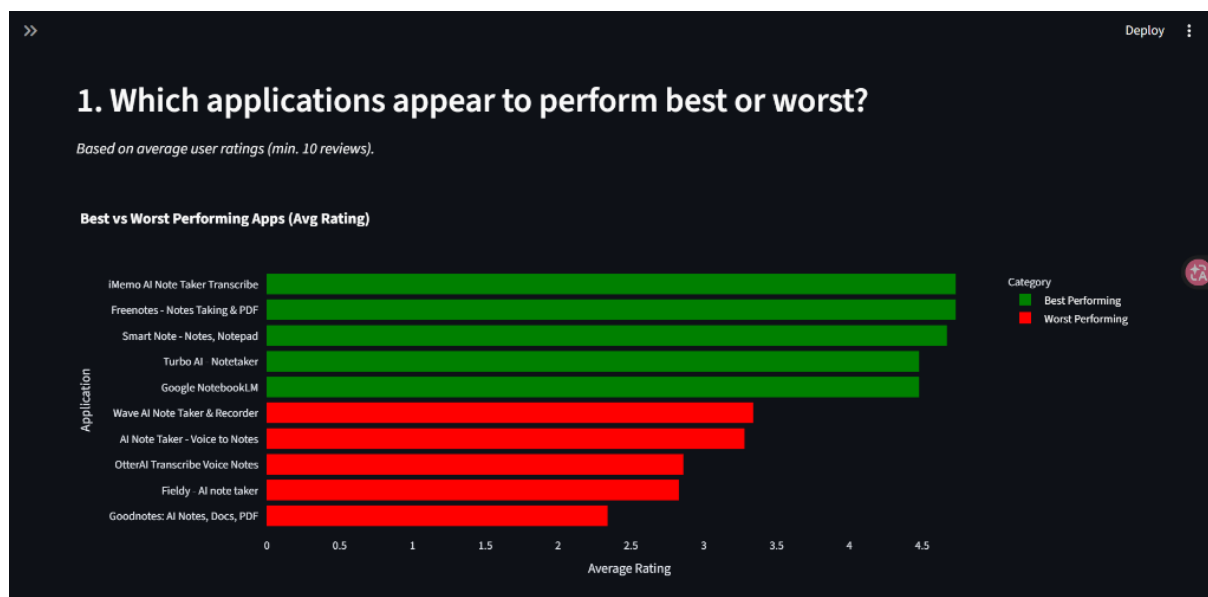


FIGURE 6 – Dashboard — Q1 : Best vs. Worst performing apps by average rating

2.5.3 Rating Trend Over Time (Q2)

The daily average rating time series is enriched with a **30-Day Simple Moving Average (SMA-30)**, which smooths daily fluctuations and reveals the underlying trajectory of user satisfaction.

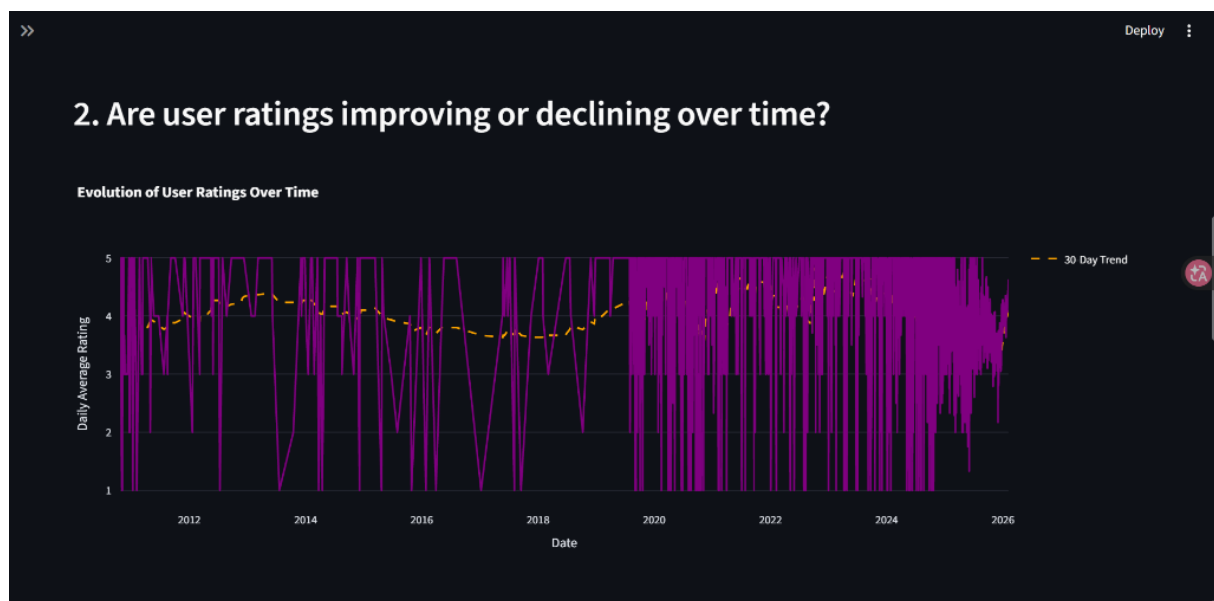


FIGURE 7 – Dashboard — Q2 : Rating evolution with SMA-30 trend line

2.5.4 Review Volume Distribution (Q3)

The third chart presents the distribution of review volume per application (Top 20), highlighting the sometimes significant disparities in visibility between apps on the platform.

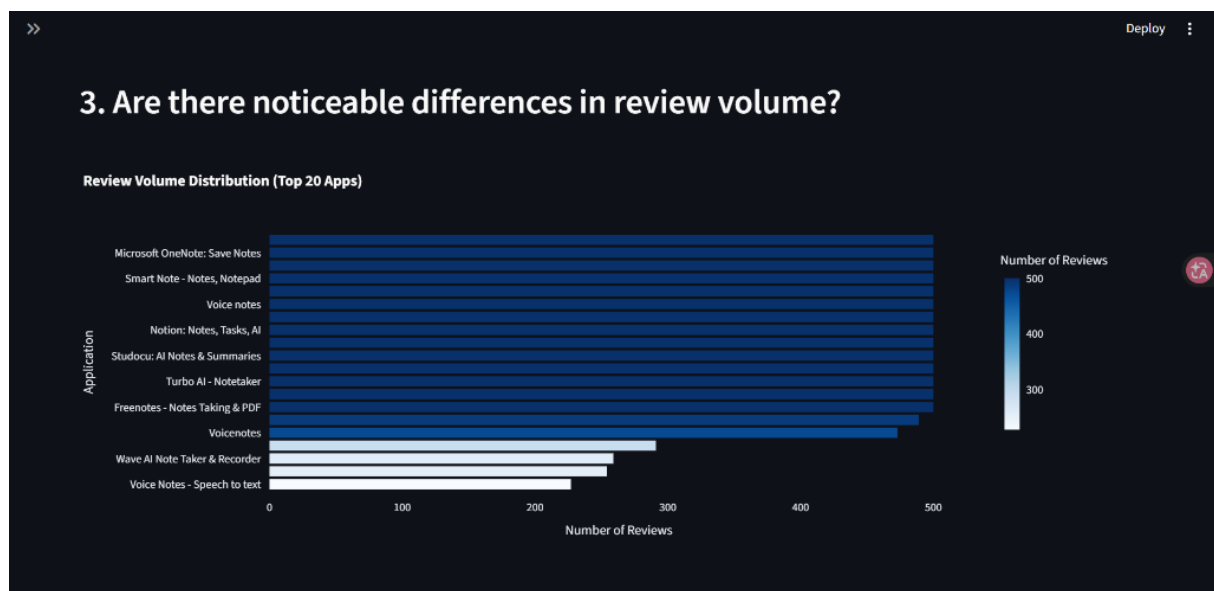


FIGURE 8 – Dashboard — Q3 : Review volume distribution across applications

2.6 Limitations of this Approach

While functionally complete and analytically relevant, this Python-based pipeline reveals structural weaknesses that motivate the transition to a more robust architecture in Phase 2.

Why migrate to dbt? **Manual orchestration** : scripts must be run in a strict order with no built-in dependency tracking. A failure in an upstream step does not automatically halt downstream processing.

Memory bottleneck : `pandas` loads the entire dataset into RAM. On larger volumes, this in-memory processing model would quickly hit physical limits.

No state management (Full Refresh) : every pipeline run reprocesses all historical data from scratch, even if only a few thousand new reviews have been added since the last run — highly inefficient for production use.

3 Phase 2 : Modern ELT with dbt & DuckDB

3.1 From ETL to ELT : A Paradigm Shift

The fundamental limitation of Phase 1 was that all transformation logic lived inside Python scripts consuming the server's RAM. In Phase 2, we adopted an **ELT (Extract, Load, Transform)** paradigm : raw JSON files are read *directly* by **DuckDB** at query time, and all transformations are expressed as SQL models orchestrated by **dbt** (data build tool). This shift eliminates the memory bottleneck of in-memory pandas processing entirely, moves computation to where the data resides, and enables a declarative, version-controlled, and testable approach to data modeling.

Technology Stack **DuckDB** is an in-process analytical database engine, capable of reading JSON and JSONL files natively via its `read_json_auto()` function, without any prior ingestion step. **dbt** (data build tool) is a transformation framework that compiles SQL **SELECT** statements into executable database objects (views or tables), manages model dependencies through a directed acyclic graph (DAG), generates interactive documentation, and enforces data quality through a built-in testing framework.

3.2 Data Architecture : From Star to Snowflake Schema

The initial design considered a classic **Star Schema** : a single central fact table (`fact_reviews`) connected to flat dimension tables. While simple to implement, a pure Star Schema would duplicate developer contact details and genre labels across every row of `dim_apps`. We therefore normalized the design into a **Snowflake Schema** by extracting these shared attributes into dedicated lookup tables, reducing redundancy and enabling foreign key constraints.

The DAG below, generated by `dbt docs serve`, shows the full dependency chain from raw sources to the final mart models.

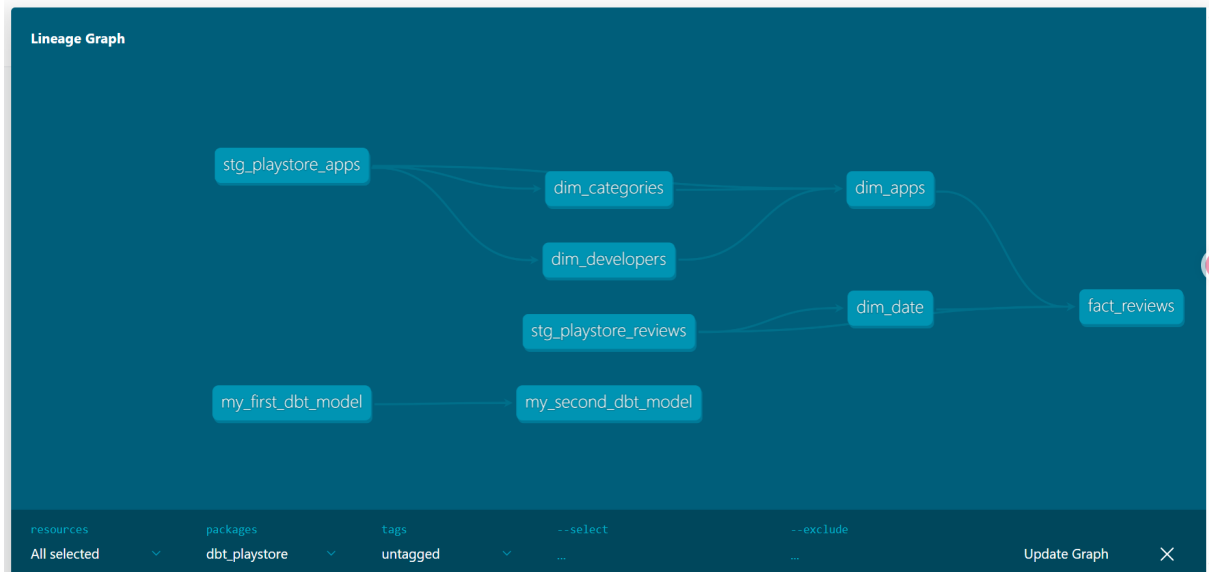


FIGURE 9 – dbt Lineage Graph (DAG) — full model dependency chain generated by dbt docs serve

3.3 Staging Layer

The staging layer is the single point of contact with raw data. Both models use DuckDB's `read_json_auto()` to read files directly from disk, applying only column renaming and type normalization, with no business logic.

`stg_playstore_apps` reads `apps_metadata.json` and exposes 15 standardized fields, including `app_id`, `developer_email`, `developer_website`, `genre_id`, and `updated_timestamp`.

`stg_playstore_reviews` reads `apps_reviews.jsonl` and maps 9 fields including `review_id`, `thumbs_up_count`, and `review_timestamp`. The timestamp field is preserved as-is so it can be used by the incremental filter downstream.

3.4 Mart Layer : Dimensional Models

3.4.1 Normalized Dimension Tables

`dim_developers` and `dim_categories` are derived from `stg_playstore_apps` using `SELECT DISTINCT` to deduplicate values. Each table uses `ROW_NUMBER() OVER (ORDER BY ...)` to generate a sequential integer surrogate key, which is a standard data warehousing practice to replace natural string identifiers with lightweight integer foreign keys.

`dim_apps` is the most complex dimension. It reads from the `apps_snapshot` table (not from staging) to expose the full historical record of each application, then joins `dim_developers` and `dim_categories` to retrieve their surrogate keys. It also derives the boolean column `is_paid` via a `CASE WHEN price > 0` expression.

`dim_date` generates a complete calendar from the minimum to the maximum review date using DuckDB's native `range()` function. It exposes `date_key` (an integer in YYYYMMDD format for fast join performance), `year`, `month`, `day`, `quarter`, `day_of_week`, and the boolean `is_weekend`, derived from DuckDB's `dayofweek()` function.

3.4.2 Fact Table : fact_reviews

The central fact table joins all three dimension tables and records one row per user review. It contains the following foreign keys : `app_key` (joining `dim_apps`), `developer_key` (joining `dim_developers`), and `date_key` (joining `dim_date`). The review payload columns (`rating`, `thumbs_up_count`, `review_text`, `review_version`, `review_timestamp`) provide the measurable facts of each transaction.

3.5 Features Added Beyond Base Requirements

Beyond the core requirements of implementing SCD2, Incremental Loading, and Data Quality tests, we introduced several additional design decisions that strengthen the pipeline.

Additional Contributions Normalized Snowflake Schema : Rather than stopping at a flat Star Schema, we normalized developer and category data into dedicated `dim_developers` and `dim_categories` tables. This reduces data duplication and makes the schema extensible to new attributes without modifying the fact table.

Integer Surrogate Keys : All dimension primary keys (`app_key`, `developer_key`, `category_key`, `date_key`) are generated as sequential integers using `ROW_NUMBER()` `OVER (ORDER BY ...)`, following data warehousing best practices for join performance and decoupling from source system identifiers.

Enriched Date Dimension : `dim_date` was built using DuckDB's native `range()` function to generate a full date spine. It includes `quarter`, `day_of_week`, and the boolean `is_weekend` column for richer time-based analysis beyond simple date filtering.

Derived Business Metric : The `is_paid` boolean in `dim_apps` (derived as `CASE WHEN price > 0`) adds an analytical dimension that allows segmenting free versus paid application performance without modifying the fact table.

3.6 Advanced Features

3.6.1 Slowly Changing Dimensions Type 2 — SCD2

A core requirement of production data warehousing is the ability to track historical changes in dimension attributes over time. Application prices and genre classifications on the Play Store can change, and a simple overwrite strategy would permanently destroy that historical information. We addressed this using **dbt Snapshots**, configured in `snapshots/apps_snapshot.sql`.

Snapshot Configuration (snapshots/apps_snapshot.sql)

```

1 {% snapshot apps_snapshot %}
2 {{
3     config(
4         target_schema='snapshots',
5         unique_key='app_id',
6         strategy='check',
7         check_cols=['price', 'genre', 'app_name']
8     )
9 }}
10 select * from {{ ref('stg_playstore_apps') }}
11 {% endsnapshot %}

```

The `strategy='check'` option instructs dbt to compare the values of the three monitored columns on each run. Whenever a change is detected, the current record's `dbt_valid_to` is stamped with the current timestamp and a new row is inserted. The current state of any application is always retrieved by filtering on `dbt_valid_to IS NULL`.

FIGURE 10 – DuckDB query result confirming `dbt_valid_from` and `dbt_valid_to` columns in the snapshot table — [screenshot to add](#)

3.6.2 Incremental Loading — `fact_reviews`

The `fact_reviews` table accumulates reviews continuously and is by far the largest table in the warehouse. A full refresh on every run would reprocess all historical records unnecessarily. Instead, the model is configured with **Incremental Materialization**: on any run that is not a forced full refresh, dbt wraps the query with the `is_incremental()` macro, which appends a `WHERE` clause that retains only records newer than the most recent `review_timestamp` already present in the table.

Incremental Filter (models/marts/fact_reviews.sql)

```

1 {{ config(materialized='incremental', unique_key='review_id') }}
2
3 ...
4 from stg_reviews
5 inner join dim_apps on stg_reviews.app_id = dim_apps.app_id
6 inner join dim_date on stg_reviews.review_timestamp::date = dim_date.date
7
8 {% if is_incremental() %}
9     AND stg_reviews.review_timestamp > (
10         SELECT MAX(review_timestamp) FROM {{ this }}
11     )
12 {% endif %}

```

FIGURE 11 – Terminal output of `dbt run` in incremental mode showing near-instant execution — [screenshot to add](#)

3.7 Data Quality Enforcement

All 7 models are covered by declarative quality tests defined in `schema.yml` files co-located with each model layer. The tests are automatically executed on every `dbt test` run and fail the pipeline immediately if any assertion is violated.

Test Coverage Across All Models Every primary key (`app_key`, `developer_key`, `category_key`, `date_key`, `review_id`) is verified for **uniqueness** and **non-nullability**. Every foreign key relationship is enforced through **relationship tests** : `dim_apps.developer_key` is validated against `dim_developers`, `dim_apps.category_key` against `dim_categories`, and all three foreign keys of `fact_reviews` (`app_key`, `developer_key`, `date_key`) are validated against their respective dimension tables.

4 Comparative Analysis & Critical Reflections

4.1 Python-Only vs dbt-Based Pipeline

Both Phase 1 and Phase 2 answer the same business questions, but they do so through fundamentally different engineering philosophies. In Phase 1, the developer is responsible for everything : explicitly coding each transformation step, manually ordering script execution, loading data into RAM, and writing separate scripts to compute even simple aggregations. This approach works for small, one-off projects, but it introduces hidden fragility at every layer, from the order scripts are run to the size of the machine's available memory.

In Phase 2, **dbt** shifts the developer's role from "writing procedures" to "describing outcomes." A model in dbt is nothing more than a **SELECT** statement describing what a table should contain. dbt figures out how and when to build it. Combined with **DuckDB**'s ability to query JSON files natively, the pipeline becomes stateless, testable, and self-documenting in a way that a collection of Python scripts simply cannot be.

The table below summarizes the key differences across five engineering dimensions :

Criterion	Python Pipeline (Phase 1)	dbt Pipeline (Phase 2)
Paradigm	Imperative (defines <i>how</i> to transform)	Declarative (defines <i>what</i> the result should be)
State Management	No state — every run is a full refresh of all data	Built-in via Snapshots (SCD2) and Incremental models
Performance	Memory-bound : pandas loads entire datasets into RAM	Compute-bound : DuckDB processes data in-engine, close to the file
Testing	No automated data tests; correctness relies on the developer	Declarative YAML tests run automatically on every dbt test call
Documentation	External and quickly outdated as code evolves	Auto-generated from the code itself; always up to date with the DAG

TABLE 1 – Comparison of the Python-only pipeline (Phase 1) and the dbt-based pipeline (Phase 2)

4.2 Critical Reflections

4.2.1 Most Fragile Part of the Pipeline

Raw Input Schema Has No Contract The most fragile component of the entire pipeline is the boundary between the Google Play Store API and our staging layer. Our `stg_playstore_apps` model references specific field names such as `realInstalls`, `developerEmail`, and `genreId` that are returned by the `google_play_scraper` library. If the Play Store updates its response format or the library changes its field mappings, every downstream model — from `dim_apps` to `fact_reviews` — would silently break or produce NULL values with no automatic alert. The absence of a schema contract at ingestion is the single greatest risk to pipeline reliability.

4.2.2 Biggest Architectural Insight

The most significant insight gained through this project is that **data modeling is fundamentally a dependency graph problem**, not a scripting problem. In Phase 1, the execution order of `transform_data.py` and `create_serving_layer.py` was an implicit, invisible convention maintained entirely by the developer. In Phase 2, that same dependency is made explicit through every `ref()` call in our SQL models : `fact_reviews` references `dim_apps`, which references `apps_snapshot`, which references `stg_playstore_apps`. dbt resolves this chain automatically, parallelizes models with no shared dependencies, and refuses to continue if any upstream model fails. What was once a fragile convention becomes a machine-enforced, visually inspectable execution plan — the DAG.

4.2.3 One Design Decision We Would Change

If we were to rebuild the pipeline, we would fundamentally rethink how `dim_date` is generated. In the current implementation, the date dimension is constructed using DuckDB's `range()` function bounded by the minimum and maximum `review_timestamp` values found in the reviews table. This means that any date for which no review exists in the dataset is simply absent from the calendar, creating silent gaps in any time-series analysis. A production-grade approach would generate a fixed, complete calendar spanning 10 to 20 years, entirely independent of transaction data, guaranteeing that every date can always be joined against without missing rows.

5 Conclusion

This project successfully demonstrated the complete evolution of a data engineering workflow across two distinct phases. Starting from a collection of hand-written Python scripts that extracted, cleaned, and visualized Google Play Store data, we progressively identified the structural limitations of that approach and rebuilt the pipeline around a modern, declarative ELT architecture.

The final system, built on `dbt` and `DuckDB`, delivers a normalized Snowflake Schema composed of five interdependent models. It incorporates production-grade engineering patterns that go well beyond a basic implementation : `SCD2 Snapshots` preserve the full history of application metadata changes ; `Incremental Loading` on `fact_reviews` ensures that only new records are processed on each run, drastically reducing execution time ; and a comprehensive suite of `data quality tests` declared in `schema.yml` automatically validates every primary key and foreign key relationship across all seven models.

Perhaps most importantly, the project demonstrated that the shift from imperative scripting to declarative modeling is not merely a technical change. It is a change in how a data team thinks about their work : from writing procedures to defining contracts.

Future Work Several enhancements could further industrialize this pipeline. Wrapping the `dbt` execution in an orchestrator such as `Airflow` or `Dagster` would replace the current manual `dbt run` commands with scheduled, dependency-aware DAGs that can trigger alerts on failure. Integrating a CI/CD pipeline via `GitHub Actions` would automate `dbt test` on every pull request, preventing regressions from reaching production. Finally, replacing the data-bounded `dim_date` with a fixed, pre-generated calendar table would eliminate the gaps in time-series analysis identified in the reflections above, making the analytical layer fully robust for any future date range.

A Project Evidence

A.1 SCD2 Verification

FIGURE 12 – Result of SCD2 Snapshot query showing history tracking
(dbt_valid_from/to)

A.2 Incremental Loading Log

FIGURE 13 – Terminal output showing fast execution of incremental model